

Preguntas y respuestas — Demo Harness

1. ¿Debo borrar todo /src menos el README?

En esta copia local ya está así — /src solo tiene README.md, nada más. Así tiene que estar **antes** de arrancar la demo, para que la audiencia vea el rojo real y después vea a Copilot construirlo en vivo. Si en la PC del trabajo ya probaste el flujo y quedó algo implementado ahí, sí: borra todo dentro de src/ excepto README.md antes de presentar (o hacé `git checkout HEAD -- src && git clean -fd src` si esa carpeta también es un repo git).

2. ¿La demo en vivo es correr simplemente el texto que me diste en Copilot?

No es *solo* eso — son 3 acciones en vivo, en este orden:

- (a) Correr `verify.ps1` con `/src` vacío → mostrar el rojo.
- (b) Pegar el prompt en Copilot (modo Agent) → dejarlo iterar hasta verde.
- (c) Inyectar la violación a mano, correr `verify.ps1` de nuevo → mostrar que la caza.

El prompt a Copilot es la parte (b), la más larga, pero no es toda la demo.

3. ¿Qué es feedforward?

Es todo lo que le das al agente **antes** de que escriba una línea de código, para guiarlo. En este caso: `specs/constitution.md` y `specs/products-spec.md`. Es "hacia adelante" porque actúa antes del hecho, previniendo — le decís las reglas de antemano en vez de corregir después. Se complementa con el **feedback**, que es lo que verifica *después* (los gates). Harness = feedforward + feedback trabajando juntos.

4. ¿Qué hacen `verify.ps1` y `verify.sh`?

Son el mismo script, escrito dos veces para dos sistemas operativos (`.ps1` = Windows/PowerShell, `.sh` = Linux/Mac/bash) — se usa `verify.ps1` en Windows. Ambos hacen lo mismo:

1. Compilan `/gates`.
2. Si `src/Api/Api.csproj` ya existe, levantan la API en `localhost:5087`.
3. Corren los 18 tests contra esa API.
4. Apagan la API.
5. Devuelven exit code 0 (verde) o distinto de 0 (rojo).

5. ¿Qué hacen los gates? ¿Son el harness?

Los gates son los chequeos automáticos: 13 tests que le pegan por HTTP a la API para validar comportamiento, y 5 reglas que inspeccionan el código compilado para validar que se respeta la arquitectura en capas. Deciden si el código "entra" o no. **Son el feedback, no todo el harness.** El harness completo = specs (feedforward) + gates (feedback) + el script que los conecta (el loop). Los gates son "el corazón" en el sentido de que son la parte escrita con más cuidado, pero sin las specs no tendrían qué hacer cumplir.

6. Paso C — ¿cómo inyecto la violación exactamente?

Abrir `src/Api/Controllers/ProductsController.cs` (va a existir recién después del paso B, cuando Copilot ya implementó todo) y agregar esto:

```
```csharp using Microsoft.EntityFrameworkCore; // <- agregar este using arriba

public sealed class ProductsController : ControllerBase { private readonly IProductService _productService; private readonly DbContext _db = null!;
// <- agregar este campo ... ```
```

Si al compilar tira error porque `Api.csproj` no tiene el paquete de EF Core, agregar esta línea dentro de un `<ItemGroup>` en `src/Api/Api.csproj`:

```
xml <PackageReference Include="Microsoft.EntityFrameworkCore"
Version="8.0.10" />
```

Después correr `.\scripts\verify.ps1` de nuevo.

### Qué se va a ver fallar: el test

`ARCH01_Api_DoesNotDependOn_EntityFrameworkCore`, con un mensaje que dice `Failing types: Api.Controllers.ProductsController` — señala el nombre exacto de la clase culpable. El resto (17 de 18) va a seguir en verde.

Después borrar esas dos líneas agregadas (y la del `csproj` si se agregó) y correr `verify.ps1` una vez más para cerrar en verde de nuevo.

## 7. Las specs están en español pero la demo es en inglés — ¿las dejo así?

Sí, no hace falta traducirlas. Algunas razones:

- Lo que realmente importa desde el punto de vista técnico (nombres de clases, endpoints, `Product`, `Price`, `Stock`, los tests en `/gates`) ya está en inglés porque es código C#. Lo que está en español es solo la prosa explicativa de las specs.
- El guion (`GUION.md` / `guion-eng.md`) ya prevé parafrasear cada sección en inglés mientras se muestra en pantalla — la audiencia no necesita leer el español, lo escucha explicado. Eso ya cubre el gap.
- Encaja con el mensaje de "minimalista, no me fui por las ramas" — traducir 3 documentos que no cambian el mecanismo del harness sería tiempo mal gastado.

Consejo práctico: al mostrar `constitution.md` o `products-spec.md` en pantalla, señalar 2-3 líneas puntuales y traducirlas en el momento ("acá dice que `Price` tiene que ser mayor a cero...") para que quede claro que no se esconde nada, simplemente se eligió el idioma de trabajo por velocidad.

## 8. El contenido de `gates` ¿cómo apareció? ¿Está muy atado a este proyecto o podría ser genérico?

### Cómo apareció

No se generó en esta conversación. Viene de una sesión anterior (documentada en `CHAT.md` y en la sección 5 de `harness-sdd-8090.md`): se le dio a Claude Code un prompt bastante detallado pidiendo específicamente esta estructura — un proyecto xUnit con casos derivados de `verification.md`, reglas `NetArchTest` que hagan cumplir la `constitution` (controllers sin EF Core directo, Domain sin Infrastructure, naming de controllers), y un `.editorconfig` con analyzers en `warning-as-error`. El agente de esa sesión tomó esas instrucciones y las convirtió en el código concreto (los 18 tests, `AssemblyLoader.cs`, etc.) — el prompt dio el *qué*, el agente resolvió el *cómo*. Después, en el ensayo de esta semana, se encontraron y arreglaron 2 bugs sobre ese código heredado (`AssemblyLoader` cargando `ref assemblies`, y el `Async suffix` en el controller).

### ¿Genérico o atado a este proyecto?

Se divide limpio en dos partes:

**Genérico (reutilizable en cualquier proyecto .NET con esta convención de capas):**

- `AssemblyLoader.cs` — recibe un nombre de proyecto como string, cero conocimiento de "Products".
- Las 5 reglas de `ArchitectureGateTests.cs` (`ARCH01..05`) — solo referencian nombres de ensamblado (`Api`, `Domain`, `Infrastructure`) y la convención de naming de controllers. Servirían igual para `Tax Opinion`, `Tax Apportionment`, o cualquier feature, siempre que se mantenga la misma convención `Api/Application/Domain/Infrastructure`.



- `.editorconfig` — reglas de naming/estilo puras de C#, sin ningún acoplamiento a dominio.
- El patrón de `ApiFixture.cs` + tests *black-box* por HTTP (sin `ProjectReference` a `/src`) — técnica genérica para que los gates compilen sin que la app exista todavía.
- `verify.ps1 / .sh` — el loop de build+test+exit-code, cero conocimiento de dominio.

### **Específico de este proyecto (Products CRUD):**

- Solo `ProductsApiGateTests.cs` — los 13 tests AC-01..13 están escritos contra `/api/products`, con el shape exacto de `Product` (Name/Price/Stock) y sus reglas de negocio puntuales.

Y eso es esperable, no un defecto: los tests funcionales tienen que reflejar la spec de la feature 1:1 — si mañana cambia el dominio (por ejemplo a Tax Opinion), se reescribe `verification.md` + ese único archivo de tests, y el resto de `/gates` se reutiliza sin tocar una línea.